

CHAPTER 2: SURVEY OF TECHNOLOGIES

2.1 Overview

The design of DrishtiX v3.0 is governed by a single, non-negotiable architectural constraint: **every computational operation — from raw pixel capture to annotated alert delivery — must execute locally on a standard laptop, without any cloud dependency, within a latency envelope of 200 milliseconds.**

This constraint eliminates entire categories of technology choices that dominate conventional surveillance platforms (cloud-hosted inference APIs, browser-rendered dashboards, centralized SQL warehouses) and demands a stack that is simultaneously high-performance, memory-efficient, and thermally sustainable across 8–12-hour operational shifts.

This chapter presents a rigorous survey of the technologies selected for the DrishtiX v3.0 stack, organized by architectural layer. For each technology, we examine: (a) the specific problem it solves within the pipeline, (b) the alternatives that were evaluated and rejected, and (c) the technical justification for the final selection. The survey proceeds bottom-up — from the deep learning inference layer, through the persistence and concurrency layers, to the presentation and external integration layers.

2.2 Computer Vision and Deep Learning Inference

2.2.1 The Legacy Pipeline: Haar Cascade + LBPH

The DrishtiX v1.0 and v2.0 releases employed the classical two-stage computer vision pipeline that has served as the entry point for face recognition systems since the early 2000s:

- **Detection Stage — Viola-Jones Haar Cascade Classifier:** The `haarcascade_frontalface_alt2.xml` classifier, shipped with OpenCV, uses a cascade of boosted weak classifiers trained on Haar-like rectangular features. While computationally inexpensive (~5–10 ms per frame), the Haar Cascade suffers from three fundamental limitations in operational surveillance contexts:
 1. **Pose Sensitivity:** The classifier is trained exclusively on near-frontal face orientations. Profile views, head tilts exceeding $\pm 15^\circ$, and downward-looking subjects produce systematic detection failures.
 2. **Illumination Fragility:** Haar features encode contrast gradients between adjacent rectangular regions. Non-uniform lighting — overhead fluorescents casting hard chin shadows, backlighting from windows, low-light corridors — disrupts these gradients and collapses detection accuracy below operationally useful thresholds.
 3. **Single-Face Bias:** While technically capable of multi-face detection, the classifier's sliding-window approach combined with conservative non-maximum suppression tuning produces severe performance degradation beyond 3–4 simultaneous faces per frame.
- **Recognition Stage — Local Binary Pattern Histograms (LBPH):** The LBPH recognizer computes statistical histograms of local texture micro-patterns across an 8-neighbourhood circular radius. While LBPH offers modest illumination invariance (relative to raw pixel comparison or Eigenfaces), it suffers from a critical architectural limitation: **it is a holistic texture encoder, not a geometric identity**

encoder. Any occlusion of the lower face — surgical masks, scarves, hands — destroys the histogram distribution and renders matching unreliable. Furthermore, LBPH requires **full model retraining** whenever a new target is added to the watchlist, a process that is both computationally expensive and operationally disruptive.

Retention as Fallback: Despite these limitations, the Haar+LBPH pipeline is retained in v3.0 as a zero-degradation fallback mechanism, selectable via a single configuration key (`face_detection_method = "HAAR"`) in MongoDB. This ensures operational continuity on legacy field hardware that lacks ONNX model execution support.

2.2.2 The DNN Pipeline: YuNet + SFace (ONNX)

The v3.0 architecture replaces the legacy pipeline with production-grade deep neural network models distributed in the **ONNX (Open Neural Network Exchange)** format, executed via OpenCV's built-in DNN module (`opencv_dnn`). The ONNX format — a vendor-neutral, hardware-agnostic serialization standard — was selected specifically because it decouples model training frameworks (PyTorch, TensorFlow) from the inference runtime, enabling DrishtiX to execute models without requiring Python, TensorFlow, or CUDA to be installed on the target machine.

2.2.2.1 Face Detection: FaceDetectorYN (YuNet)

Model: `face_detection_yunet_2023mar.onnx`

Architecture: YuNet is a lightweight convolutional neural network designed for real-time multi-face detection. It employs a Feature Pyramid Network (FPN) backbone that processes the input frame at multiple spatial scales simultaneously, enabling detection of faces ranging from 80×80 pixels (distant subjects in a corridor) to full-frame close-ups.

Why YuNet over Alternatives:

Criteria	Haar Cascade	MTCNN	RetinaFace	YuNet
Inference Latency (CPU)	5–10 ms	40–80 ms	60–120 ms	~2 ms
Multi-Face (10+)	Poor	Good	Excellent	Excellent
Pose Tolerance	±15°	±30°	±45°	±45°
5-Point Landmarks	✗	☑	☑	☑
ONNX Support	N/A	Partial	Partial	Native
Thermal Sustainability	☑	✗ (GPU)	✗ (GPU)	☑

YuNet's ~2 ms inference latency is critical to the DrishtiX architecture: it allows detection to execute **on the main capture thread** without frame drops, reserving the dedicated thread pools for the heavier recognition stage.

Output Contract: For each detected face, YuNet returns:

- Bounding box coordinates (`x, y, width, height`)
- Detection confidence score (0.0–1.0)

- Five facial landmark points: left eye, right eye, nose tip, left mouth corner, right mouth corner

The 5-point landmarks are indispensable for the downstream **face alignment** step, where the detected face crop is geometrically normalized (affine-warped to a canonical 112×112 frontal pose) before being fed to the SFace recognition model.

2.2.2.2 Face Recognition: FaceRecognizerSF (SFace / ArcFace)

Model: `face_recognition_sface_2021dec.onnx`

Architecture: SFace (ShuffleFace) is a deep metric-learning model based on the ShuffleNet backbone, trained with an ArcFace-variant angular margin loss function. It projects aligned 112×112 face images into a **128-dimensional L2-normalized embedding space**, where identity is encoded as geometric direction rather than raw pixel similarity.

The Metric-Learning Paradigm Shift: The transition from LBPH to SFace represents a fundamental architectural paradigm shift in how identity matching is performed:

Aspect	LBPH (Legacy)	SFace (v3.0)
Representation	Texture histogram	128-dim float vector
Matching	Chi-squared distance	Cosine similarity
Adding New Target	Full model retrain	Store one vector
Occlusion Handling	Catastrophic	Geometry-resilient
Gallery Scalability	O(n) retrain	O(1) insert
Operational Threshold	Empirical	0.363 (published)

Cosine Similarity Matching: Identity matching in v3.0 is performed by computing the cosine similarity between the probe embedding (extracted from the live frame) and each gallery embedding (pre-computed at registration time and stored in MongoDB). The published operational threshold for SFace is **0.363** — any match exceeding this value constitutes a positive identification. DrishtiX stores this threshold as a configurable parameter (`dnn_cosine_threshold`) in the configuration collection, allowing field operators to tune sensitivity without code changes.

Thread Safety via ThreadLocal: The SFace `FaceRecognizerSF` instance is **not thread-safe** — concurrent calls from multiple recognition threads would produce data races on internal buffers. DrishtiX solves this via a `ThreadLocal<FaceRecognizerSF>` pattern in `DnnFaceRecognitionService`: each of the 4 recognition pool threads lazily instantiates its own private model instance, enabling true lock-free parallel embedding extraction.

Gallery Architecture: The in-memory embedding gallery is implemented as a `ConcurrentHashMap<Integer, TargetEmbeddings>` protected by a `ReentrantReadWriteLock`. Concurrent reads (matching during recognition) are fully non-blocking; writes (gallery updates during target registration) acquire the write lock and serialize. Each target stores multiple template embeddings (up to 5 photos per target) and a pre-computed centroid embedding for fast single-vector matching.

2.2.2.3 Body Re-Identification: OSNet (x0.25, MSMT17)

Model: `osnet_x0_25_msmt17.onnx`

Architecture: OSNet (Omni-Scale Network) is a person re-identification model designed to extract **512-dimensional L2-normalized appearance embeddings** from full-body or torso crops. The x0.25 variant is a quarter-width version optimized for edge deployment, trained on the MSMT17 multi-camera person re-identification dataset.

The Body Lock Problem: A critical failure mode in face-only surveillance is the **"head-turn dropout"**: when a recognized target turns their head, walks away from the camera, or is momentarily occluded by another pedestrian, the face is lost and the system generates no further alerts until the target turns back. During this gap, the suspect could exit the surveilled area entirely.

DrishtiX v3.0 addresses this through a **Facial-to-Spatial Handoff** mechanism:

1. When a face is positively identified, the bounding box is expanded downward by a factor of 2.0× to capture the upper torso/shirt region.
2. An OpenCV object tracker (KCF or CSRT) is initialized on this expanded region.
3. OSNet extracts a 512-dim body appearance embedding from the torso crop.
4. When the face is subsequently lost (head turn, occlusion), the tracker continues following the body, and OSNet periodically re-verifies identity by comparing the current torso embedding against the stored reference.
5. A configurable fusion formula blends face and body confidence: $\alpha \cdot \text{face_conf} + \beta \cdot \text{body_conf}$, with adaptive weight shifting as the face transitions from visible → grace period → body-only tracking.

This architecture allows DrishtiX to maintain persistent target locks for up to **60 seconds** (900 frames at 15 FPS) after facial loss, dramatically extending operational coverage.

2.3 Inter-Frame Object Tracking

2.3.1 KCF and CSRT Trackers

Running full YuNet + SFace inference on every captured frame would be computationally wasteful — YuNet detection alone consumes ~2 ms, but the downstream face alignment, SFace embedding extraction, and gallery matching add substantial latency per detected face. DrishtiX v3.0 employs a **frame-skipping strategy**: full DNN inference executes on every Nth frame (default: every 2nd frame), while intermediate frames are handled by lightweight OpenCV object trackers:

- **KCF (Kernelized Correlation Filter):** ~0.3 ms per tracker update. Preferred for edge laptop deployments where thermal headroom is constrained. KCF models the target appearance as a circulant matrix in the frequency domain, enabling real-time position extrapolation at negligible CPU cost.
- **CSRT (Channel and Spatial Reliability Tracker):** ~1.5 ms per tracker update. Available as a higher-accuracy alternative when the hardware permits. CSRT uses spatial reliability maps to handle partial occlusion and boundary effects more gracefully than KCF.

The tracker type is configurable at runtime via the `tracker_type` configuration key, defaulting to **"KCF"**.

2.4 Platform Language and Runtime

2.4.1 Java 17 LTS

DrishtiX is built on **Java 17 (Long-Term Support)**, the most recent LTS release at the time of development. The selection of Java as the platform language — over Python (the dominant language in the computer vision community) — was driven by three critical requirements:

- 1. **Deterministic Concurrency Primitives:** Java's `java.util.concurrent` package provides battle-tested concurrency abstractions — `CompletableFuture`, `ExecutorService`, `ReentrantReadWriteLock`, `ConcurrentHashMap`, `AtomicInteger`, `AtomicBoolean`, `AtomicReference`, `AtomicLong` — that enable the construction of a provably correct multi-threaded pipeline without the Global Interpreter Lock (GIL) limitations that plague CPython.
- 2. **JavaFX Desktop Integration:** Java is the only mainstream language that provides a first-party, hardware-accelerated desktop GUI toolkit (JavaFX) with native FXML declarative layout support, CSS-based theming, property binding, and a single-threaded UI contract (`Platform.runLater()`) that eliminates an entire class of rendering race conditions.
- 3. **JVM Memory Management:** The JVM's generational garbage collector, combined with explicit `Mat.release()` calls for native OpenCV memory through JavaCV, provides a dual-layer memory management strategy that prevents the unbounded native memory leaks characteristic of long-running OpenCV applications in Python.

2.4.2 JavaCV 1.5.10 / OpenCV 4.9.0

JavaCV serves as the Java binding layer to the native OpenCV C++ library, providing:

- Direct access to `FaceDetectorYN` (YuNet) and `FaceRecognizerSF` (SFace) via the `opencv_objdetect` module
- Raw DNN inference via `opencv_dnn.Net` for OSNet body re-identification
- Frame capture via `OpenCVFrameGrabber` and `VideoCapture`
- Object tracking via `TrackerKCF` and `TrackerCSRT` from the `opencv_tracking` module
- Native `Mat` operations for image preprocessing (resize, color conversion, blob construction)

The `javacv-platform` dependency (v1.5.10) bundles pre-compiled native binaries for all major platforms (Windows x86_64, Linux x86_64, macOS ARM64), eliminating the need for manual OpenCV compilation.

2.5 User Interface: JavaFX 21.0.2

2.5.1 Why JavaFX over Web Technologies

The decision to build DrishtiX's interface as a native JavaFX desktop application — rather than an Electron wrapper, a React web dashboard, or a Python Tkinter/Qt GUI — was driven by the following requirements:

Requirement	Web (Electron/React)	Python (Tkinter/Qt)	JavaFX 21
Video Frame Rendering	Canvas/WebGL overhead	Matplotlib (slow)	Direct GPU-backed scene graph
Frame Update Rate	~10–15 FPS	~5–10 FPS	15–30 FPS (native)

Requirement	Web (Electron/React)	Python (Tkinter/Qt)	JavaFX 21
Memory Overhead	~200–400 MB (Chromium)	~50–100 MB	~80–120 MB
Thread Safety Model	Web Workers (limited)	GIL-constrained	Platform.runLater()
CSS Theming	Full CSS3	Partial	JavaFX CSS (subset)
FXML Layout	✗	✗	☑ Declarative XML

Hardware-Accelerated Rendering: JavaFX 21 uses the Prism rendering engine, which leverages Direct3D (Windows), Metal (macOS), or OpenGL (Linux) for hardware-accelerated 2D scene graph rendering. This is critical for DrishtiX: the live camera feed is rendered as an `ImageView` node in the scene graph, and frame updates at 15+ FPS require the rendering pipeline to composite each frame in under 16 ms. A browser-based approach would incur additional overhead from the JavaScript event loop, DOM diffing, and Canvas/WebGL context switching.

2.5.2 FXML and the Declarative Layout Model

All DrishtiX views are defined in **FXML** (an XML-based declarative UI language), separating layout structure from controller logic:

- `dashboard_view.fxml`: The primary surveillance view (70/30 split — camera feed + alert sidebar)
- `registry_view.fxml`: Target watchlist management with search, filter, and CRUD operations

FXML files are loaded at runtime by the `FXMLLoader`, which instantiates the scene graph and injects `@FXML`-annotated controller fields via reflection. This separation enforces a clean MVC/MVVM-inspired architecture and enables the CSS theming system to operate independently of business logic.

2.5.3 ControlsFX 11.2.1

ControlsFX is a supplementary JavaFX control library that provides the native desktop toast notification system (`Notifications.create()`) used for the bell icon and profile icon event handlers on the dashboard header. ControlsFX notifications render as lightweight, auto-dismissing overlays that do not interfere with the live camera feed or the alert sidebar.

2.6 Database and Persistence: MongoDB 5.1

2.6.1 Why MongoDB over Relational Databases

DrishtiX stores heterogeneous data: structured target metadata, variable-length float arrays (128-dim and 512-dim embedding vectors), binary image paths, timestamped detection logs, and dynamic configuration key-value pairs. A document-oriented database was selected over a relational database (MySQL, PostgreSQL) for the following reasons:

Requirement	MySQL/PostgreSQL	MongoDB 5.1
Embedding Vector Storage	BLOB or custom type	Native array field

Requirement	MySQL/PostgreSQL	MongoDB 5.1
Schema Evolution	ALTER TABLE migrations	Schema-free documents
Variable-Length Arrays	Normalized join tables	Embedded arrays
Local Deployment	Server process + config	mongod — single binary
Java Driver Maturity	JDBC + ORM overhead	mongodb-driver-sync 5.1

2.6.2 Document Schema Design

DrishtiX uses the following MongoDB collections:

Collection	Purpose	Key Fields
<code>targets</code>	Watchlist registry	<code>targetId</code> , <code>fullName</code> , <code>category</code> , <code>caseNumber</code> , <code>profileImagePath</code> , <code>isActive</code> , <code>createdAt</code>
<code>target_images</code>	Multi-photo gallery	<code>targetId</code> , <code>imagePath</code> , <code>embedding</code> (128-dim float[])
<code>person_embeddings</code>	Pre-computed embeddings	<code>targetId</code> , <code>embedding</code> , <code>modelVersion</code>
<code>detection_logs</code>	Historical detection events	<code>targetId</code> , <code>timestamp</code> , <code>confidence</code> , <code>snapshotPath</code> , <code>cameraId</code>
<code>config</code>	Runtime configuration KV	<code>key</code> , <code>value</code>
<code>audit_log</code>	Immutable action trail	<code>action</code> , <code>details</code> , <code>timestamp</code> , <code>userId</code>
<code>alert_config</code>	Per-category alert rules	<code>category</code> , <code>soundEnabled</code> , <code>telegramEnabled</code> , <code>cooldownSeconds</code>
<code>camera_sources</code>	Registered camera devices	<code>cameraId</code> , <code>name</code> , <code>uri</code> , <code>isActive</code>
<code>counters</code>	Auto-increment sequences	<code>_id</code> (collection name), <code>seq</code> (current value)

2.6.3 Connection Management

The `DatabaseManager` singleton initializes a `MongoClient` using the connection URI from `config.properties` (default: `mongodb://localhost:27017`). The MongoDB Java driver internally maintains a connection pool with keep-alive, eliminating the need for an external pool manager like HikariCP (which is designed for JDBC/SQL databases). Connection health is validated at startup via a `ping` command.

2.7 External Integration Layer

2.7.1 Telegram Bot API

DrishtiX integrates with the **Telegram Bot API** for real-time push notifications to field officers. The `TelegramAlertService` uses Java 11's built-in `java.net.http.HttpClient` (HTTP/1.1, connection pooling, 10-second connect timeout) to send multipart `sendPhoto` requests containing:

- A detection snapshot (JPEG image file)
- A formatted HTML caption with target name, category, confidence score, camera location, and timestamp

All Telegram calls are **fully asynchronous and fire-and-forget**: they execute via `CompletableFuture.supplyAsync()` on a background thread, ensuring that network latency or API unavailability never blocks the inference pipeline.

2.7.2 Background Watchlist Ingestion Engine

DrishtiX v3.0 introduces a **5-tier concurrent ingestion pipeline** that automatically populates the local watchlist from authoritative external sources:

Source	Technology	Protocol	Data Extracted
FBI Wanted API	<code>FbiWantedApiClient</code>	REST/JSON (<code>api.fbi.gov/wanted/v1/list</code>)	Name, aliases, case number, facial images
CBI Wanted List	<code>CbiWantedScraper</code>	HTML scraping via JSoup (<code>cbi.gov.in</code>)	Name, case details, photographs
TrackChild Portal	<code>TrackChildScraper</code>	HTML scraping via JSoup (<code>trackthemissingchild.gov.in</code>)	Missing child name, age, photographs

The `BackgroundIngestionEngine` orchestrates these clients on a dedicated **2-thread, low-priority scheduled executor pool** (`DrishtiX-Ingestion`), polling every 6 hours (configurable via `ingestion_interval_hours`). Anti-DDoS protections enforce a strict 2-second rate limit between HTTP requests to prevent IP blacklisting.

2.7.3 JSoup 1.17.2

JSoup is a Java HTML parser used by the CBI and TrackChild scrapers to extract structured data from government web pages. JSoup provides a jQuery-like CSS selector API (`document.select("div.wanted-card > img")`) that enables robust data extraction even from poorly structured HTML, without requiring a full headless browser (Selenium/Playwright).

2.7.4 org.json (JSON Processing)

The `org.json` library (v20240303) provides lightweight, dependency-free JSON parsing for the FBI API response payloads and the ReID service communication protocol. It was selected over heavier alternatives (Jackson, Gson) to minimize JAR size and startup overhead.

2.8 Concurrency Architecture

2.8.1 The Five-Pool Threading Model

DrishtiX v3.0 employs a strict **five-pool threading architecture**, centrally managed by the `ThreadPools` utility class. Each pool is instantiated via double-checked locking, uses daemon threads (ensuring clean JVM exit), and is named for diagnostic traceability in thread dumps:

Pool	Threads	Priority	Purpose
JavaFX Application Thread	1	Normal	UI rendering, scene graph mutations, <code>Platform.runLater()</code> callbacks
Video Inference Pool	2	Normal	Camera frame capture, YuNet face detection, tracker updates
Recognition Inference Pool	4	Normal	SFace/OSNet embedding extraction, gallery matching via <code>CompletableFuture</code>
Audio Alert Pool	1	Normal	WAV sound playback (<code>javax.sound.sampled</code>) — isolated to prevent blocking
Ingestion Pool	2	MIN	FBI/CBI/TrackChild web scraping — low priority to avoid stealing CPU from inference

Additionally, a single-thread **Scheduled Pool** handles periodic maintenance tasks (e.g., snapshot cleanup, configuration refresh).

2.8.2 Pre-Warming Strategy

A critical optimization in DrishtiX is the **recognition pool pre-warming** performed at startup. Each of the 4 recognition threads lazily creates its own `ThreadLocal<FaceRecognizerSF>` instance, which involves loading and parsing the ONNX model file (~200 ms cold-start penalty). Without pre-warming, the first face detection after launch would experience a perceptible delay. DrishtiX eliminates this by:

1. Creating a dummy 112×112 `Mat` on each pool thread
2. Running `extractEmbedding()` to force the `ThreadLocal` model to load
3. Discarding the dummy result

This ensures all 4 threads are "hot" and ready for sub-millisecond embedding extraction before the first real face appears.

2.9 Logging and Diagnostics: SLF4J 2.0 + Logback 1.5

DrishtiX uses **SLF4J** (Simple Logging Facade for Java) as the logging abstraction layer, backed by **Logback** as the concrete implementation. This combination provides:

- Structured log output with millisecond timestamps, thread names, and logger class names
- Configurable log levels per package (e.g., `DEBUG` for `com.drishtix.service`, `WARN` for `org.mongodb`)
- Automatic log file rotation to prevent disk exhaustion during extended surveillance shifts

All service classes follow the pattern `private static final Logger log = LoggerFactory.getLogger(ClassName.class)`, ensuring consistent diagnostic output across the entire

codebase.

2.10 Build and Packaging: Apache Maven

2.10.1 Multi-Module Project Structure

DrishtiX is organized as a Maven **multi-module project**:

- **drishtix-root** (parent POM): Defines the module aggregation
- **services/drishtix-app** (child module): Contains all application source code, resources, FXML layouts, CSS themes, ONNX models, and sound assets

2.10.2 Key Build Plugins

Plugin	Version	Purpose
maven-compiler-plugin	3.12.1	Compiles Java 17 source with <code>--release 17</code>
javafx-maven-plugin	0.0.8	Launches the JavaFX application (<code>com.drishtix.DrishtiXApp</code>)
maven-shade-plugin	3.5.2	Produces a fat JAR (uber-JAR) with all dependencies embedded, configured with a <code>DrishtiXLauncher</code> main class for module-system compatibility
maven-surefire-plugin	3.2.5	Executes JUnit 5 tests

2.10.3 Testing Infrastructure

Library	Version	Purpose
JUnit Jupiter	5.10.2	Unit and integration testing framework
Mockito	5.11.0	Mock object creation for service-layer isolation
TestFX	4.0.18	Automated JavaFX UI testing (headless)
Testcontainers	1.19.7	Spins up disposable MongoDB Docker containers for DAO integration tests

2.11 Technology Stack Summary

The following table provides a consolidated view of the complete DrishtiX v3.0 technology stack:

Layer	Technology	Version	Role
Language	Java (LTS)	17	Platform language and runtime

Layer	Technology	Version	Role
UI Framework	JavaFX	21.0.2	Hardware-accelerated desktop GUI
UI Extensions	ControlsFX	11.2.1	Toast notifications, advanced controls
CV Binding	JavaCV	1.5.10	Java ↔ OpenCV native bridge
CV Engine	OpenCV	4.9.0	Image processing, DNN inference, tracking
Face Detection	YuNet (ONNX)	2023-Mar	Multi-target detection (~2 ms/frame)
Face Recognition	SFace (ONNX)	2021-Dec	128-dim metric embedding extraction
Body Re-ID	OSNet x0.25 (ONNX)	MSMT17	512-dim body appearance embedding
Object Tracking	KCF / CSRT	OpenCV 4.9	Inter-frame bounding box extrapolation
Database	MongoDB	5.1.0	Document store for targets, embeddings, logs
JSON Parsing	org.json	2024-03	FBI API response deserialization
HTML Scraping	JSoup	1.17.2	CBI/TrackChild web page extraction
HTTP Client	java.net.http	Java 17	Telegram API, FBI API communication
Logging	SLF4J + Logback	2.0.12 / 1.5.3	Structured diagnostic logging
Build	Apache Maven	3.9+	Multi-module build, fat JAR packaging
Testing	JUnit 5 + Mockito + TestFX + Testcontainers	5.10 / 5.11 / 4.0 / 1.19	Unit, integration, UI, and DAO testing